COMPLETE MATHEMATICS REFERENCE

ThreadZero (AI Thread Management) | Lone Star Ledger (Cross-Ledger Anchoring)

Leon Calvin Long II | Patent Applications 19/693,343 & [LSL Provisional] | June 2026

PART 1 — THREADZERO MATHEMATICS

Patent Application No. 19/693,343 | AI Thread Management & Routing

1.1 Thread Dependency Graph

THREAD DEPENDENCY GRAPH

$$G = (V, E, W)$$

$$V = \{\tau_1, \tau_2, \ldots, \tau_n\} \text{ (thread vertices)}$$

$$E \subseteq V \times V \text{ (directed dependency edges)}$$

$$W : E \to [0, 1] \text{ (edge weight function)}$$

The foundational graph structure. Each inference thread is a vertex. Edges represent semantic or structural dependency relationships between threads. Edge weights quantify the strength of each dependency on a unit interval.

Symbol	Definition
V	Set of all active or queued inference thread vertices
E	Set of directed edges where $(\tau_i, \tau_j) \in E$ means τ_i depends on or is semantically coupled to τ_j
W	Edge weight function mapping each edge to a real value in [0,1]
τ_i	Individual inference thread i

EDGE WEIGHT FORMULA

$$W(e_{ij}) = \lambda \cdot A(s_i, s_j) + (1-\lambda) \cdot D(\tau_i, \tau_j)$$

$$A(s_i, s_j) = \cos(s_i, s_j) \text{ (semantic affinity)}$$

Edge weights combine semantic affinity (cosine similarity of thread embedding vectors) with structural dependency strength. The balance parameter λ controls the relative contribution of semantic vs structural components.

Symbol	Definition
$A(s_i,s_j)$	Semantic affinity — cosine similarity of embedding vectors s_i and s_j
$D(\tau_i,\tau_j)$	Structural dependency strength between threads τ_i and τ_j
λ	Balance parameter ∈ [0,1] — λ=1 is pure semantic, λ=0 is pure structural
s_i, s_j	Semantic embedding vectors of threads τ_i and τ_j in $\mathbb{R}^d$

1.2 Graph Filter Operator Φ^G

GRAPH FILTER OPERATOR

$$G_f = \Phi^G(G, \theta^G) = (V, E_f, W|_{E_f})$$

$$E_f = \{ e \in E : W(e) \geq \theta^G \}$$

$$\text{Filtered Queue: } Q_f = \text{TopSort}(G_f) \text{ tie-broken by } P(t,i) \downarrow$$

The filter operator prunes all edges whose weight falls below the threshold θ_G, producing a cleaner dependency graph G_f. The filtered queue Q_f is the topological sort of G_f, with ties broken by descending priority score.

▶ THEOREM 1 (Idempotency): $\Phi^G(\Phi^G(G)) = \Phi^G(G)$ □ Applying the filter twice produces the same result as applying it once.

Symbol	Definition
θ^G	Filter threshold ∈ [0,1] — edges with W(e) < θ_G are removed
E_f	Retained edge set — only edges meeting the weight threshold
G_f	Filtered graph — semantically coherent subgraph of G
Q_f	Filtered Execution Queue — topologically sorted thread dispatch order

1.3 Semantic Vector Routing Engine

SEMANTIC EMBEDDING

$$s_i = \text{Embed}(\text{context}_i) \in \mathbb{R}^d$$

Each thread's natural language context is mapped to a fixed-dimension real-valued vector using a learned transformer-based embedding function. Semantically similar threads map to geometrically proximate vectors under cosine distance.

Symbol	Definition
s_i	Semantic vector of thread τ_i in $\mathbb{R}^d$
$\text{Embed}(\cdot)$	Learned embedding function (e.g. transformer sentence encoder)
d	Embedding dimensionality — system parameter
context_i	Natural language context of thread τ_i

AGENT AFFINITY SCORE Ψ(I,J)

$$\Psi(i,j) = \cos(s_i, \text{cap}_j) \cdot (1 - L(a_j)/L_{max})^\beta$$

$$\cos(s_i, \text{cap}_j) = (s_i \cdot \text{cap}_j) / (||s_i|| \; ||\text{cap}_j||)$$

The affinity score combines semantic alignment (cosine similarity between thread vector and agent capability centroid) with a load-balancing penalty. β controls sensitivity to agent load — higher β penalizes loaded agents more aggressively.

▶ THEOREM 2 (Pareto-Optimality): The affinity function Ψ(i,j) achieves a Pareto-optimal tradeoff between semantic alignment and load balance. □

Symbol	Definition
cap_j	Capability centroid of agent a_j in $\mathbb{R}^d$ — center of its semantic specialization region
$L(a_j)$	Current load of agent a_j — number of threads assigned or queued
L_{max}	Maximum load capacity across all agents
β	Load-sensitivity exponent — β→0: pure affinity; β→∞: pure load balancing

OPTIMAL ROUTING DECISION

$$j^* = \text{argmax}_j \; \Psi(i,j)$$

$$\text{subject to: } \sigma_i \in \text{admissible\_tags}(a_{j^*})$$

$$L(a_{j^*}) \leq L_{max}$$

The optimal agent j^* maximizes the affinity score subject to two hard constraints: (1) the thread's SOT domain tag must be within the agent's declared competency vocabulary; (2) the selected agent must not be at capacity.

Symbol	Definition
j^*	Index of the optimally selected agent
σ_i	Semantic Ontology Tag of thread τ_i — its classified domain (NLP, MATH, CODE, ...)
$\text{admissible\_tags}(a_j)$	Set of SOT domain tags within the competency vocabulary of agent a_j

1.4 Execution Priority Protocol (EPP)

DYNAMIC PRIORITY SCORE P(T,I)

$$P(t,i) = w_1 \cdot U(s_i) + w_2 \cdot D\_depth(\tau_i) + w_3 \cdot (1/L(a_j))$$

$$+ w_4 \cdot (1 - e^{-\gamma(t-t_{0i})}) + w_5 \cdot C(\tau_i)$$

$$\text{constraint: } w_1 + w_2 + w_3 + w_4 + w_5 = 1$$

The priority score is a weighted linear combination of five components, each measuring a distinct dimension of urgency. Weights are calibrated from historical execution data via reinforcement learning and stored in the MS-SMT strategic context level.

▶ THEOREM 3 (Anti-Starvation): Because the aging term $(1-e^{-\gamma(t-t_0)})$ grows unboundedly toward 1, no thread τ_i can be indefinitely prevented from execution — $P(t,i) \to w_4 > 0$ as $t \to \infty$. □

Symbol	Definition
$U(s_i)$	Utility score ∈ [0,1] — estimated value of completing τ_i based on its semantic vector
$D\_depth(\tau_i)$	Dependency depth — length of longest path to τ_i in G_f; higher = more threads blocked
$1/L(a_j)$	Agent load reciprocal — higher when assigned agent is less loaded
$(1-e^{-\gamma(t-t_{0i})})$	Anti-starvation aging term — grows monotonically from 0→1 as wait time increases
$C(\tau_i)$	Criticality indicator ∈ [0,1] — application-layer urgency or real-time deadline flag
$w_1 \ldots w_5$	Weight coefficients summing to unity — trained via RL from historical performance
γ	Aging rate parameter — controls speed of anti-starvation term growth
t_{0i}	Enqueue timestamp of thread τ_i

PREEMPTION TRIGGER CONDITION

$$\text{Preempt when: } P(t, \tau\_new) > P(t, \tau_i) + \tau\_preempt$$

$$\text{Default: } \tau\_preempt = 0.50$$

When a queued or arriving thread τ_new has a priority score exceeding that of the currently executing thread τ_i by more than the preemption threshold, τ_i is checkpointed to MS-SMT and τ_new is dispatched immediately.

Symbol	Definition
$\tau\_preempt$	Preemption threshold — minimum priority differential required to trigger preemption
τ_new	Candidate preempting thread
τ_i	Currently executing thread

RL WEIGHT CALIBRATION REWARD

$$r(t) = \alpha_1 \cdot \text{CoherenceScore}(t)$$

$$+ \alpha_2 \cdot \text{CompletionRate}(t)$$

$$- \alpha_3 \cdot \text{Latency}(t)$$

The EPP weight vector $[w_1,w_2,w_3,w_4,w_5]$ is trained via a reinforcement learning agent using this reward function. Reward coefficients α_1, α_2, α_3 are domain-specific hyperparameters. Policy is stored in MS-SMT Level L-3 and loaded at inference time.

Symbol	Definition
CoherenceScore(t)	Semantic coherence quality of outputs at time t — primary positive reward
CompletionRate(t)	Fraction of threads completed without preemption or drift — efficiency signal
Latency(t)	Mean thread-to-completion time — penalized to discourage unnecessary delays
α_1, α_2, α_3	Reward weighting hyperparameters — domain-specific calibration

1.5 Dynamic Priority Thread Routing (DPTR)

SEMANTIC DRIFT METRIC Δ_S

```
Δ_s(t,i) = 1 − cos(sᵢ(t), sᵢ(t₀))
Δ_s ∈ [0, 2]
Hot-swap trigger: Δ_s(t,i) > δ
```

Semantic drift measures how far a thread's current context has moved from its original context at dispatch time. A value of 0 means no drift; 2 means maximum divergence. When drift exceeds threshold δ, the hot-swap protocol is initiated.

▶ THEOREM 4 (DPTR Termination): The DPTR hot-swap protocol terminates in at most O(k) re-routings for any thread, where k = |A| is the number of available agents, because loop detection prevents revisiting prior (thread, agent) pairs. □

Symbol	Definition
$s_i(t)$	Updated semantic vector of thread τ_i at time t — recomputed from evolving context
$s_i(t_0)$	Initial semantic vector of thread τ_i at dispatch time t_0
δ	Drift threshold — default δ=0.35 in preferred embodiment
Δ_s = 0	No semantic drift — thread context is identical to its initial state
Δ_s = 2	Maximum drift — thread context is orthogonal to its initial state

1.6 Computational Complexity Analysis

Operation	Complexity	Governing Parameters
GFTM graph construction	$O(n^2 \cdot d)$	n = active threads; d = embedding dimension
Graph filter Φ^G	$O(\|E\|)$	|E| = edges in current graph
SV routing — batch	$O(n \cdot k \cdot d)$	k = number of available agents
EPP priority update	$O(n)$	Linear in active thread count
DPTR drift monitor/token	$O(d)$	Per sampled token per active thread
MS-SMT checkpoint/restore	$O(L \cdot \log n)$	L = memory tree levels; n = context nodes
DPTR max re-routings	$O(k)$	k = |A| (loop detection bound)

1.7 Empirical Performance — 1,000-Thread Benchmark

Metric	ThreadZero	FIFO	Round-Robin	Improvement vs FIFO
Output Coherence Score	91	67	71	+36%
Execution Latency (normalized)	0.42	1.00	0.88	−58%
Thread Starvation Rate	2.1%	18.7%	12.3%	−89%

PART 2 — LONE STAR LEDGER MATHEMATICS

LSL Provisional Patent Application | Cross-Ledger Anchoring System

2.1 Universal Scribe Encoding Engine

CAPSULE CONTENT HASH

```
H_content = H(D)
uScribeId = SHA-256(capsuleBytes)
base44Id = Base44Encode(uScribeId)
```

The input data state D is hashed to produce the Capsule content hash. The complete serialized Capsule structure (including version, timestamp, creator key, hash, and flags) is then hashed to produce the 32-byte Universal Scribe Identifier (uScribeId). The uScribeId is then Base44-encoded to produce a human-readable identifier.

Symbol	Definition
`D`	Input data state — any digital artifact (document, receipt, treasury state, etc.)
`H(D)`	Cryptographic hash of D — SHA-256 in primary embodiment, SPONGE-X586 in alternative
`capsuleBytes`	Serialized binary Capsule structure: [version|hashAlg|timestamp|pubkey|H(D)|flags|extra]
`uScribeId`	32-byte SHA-256 hash of capsuleBytes — unique identifier in Universal Scribe namespace
`base44Id`	Human-readable Base44 encoding of uScribeId over 44-character URL-safe alphabet

BASE44 ENCODING SCHEME

```
Alphabet Σ₄₄ = first 44 chars of:
"ABCDEFGHJKLMNPQRSTUVWXYZ23456789
abcdefghijkmnpqrstuvwxyz"
base44Id = positional(uScribeId as BigInt, base=44, alphabet=Σ₄₄)
```

The uScribeId (interpreted as a 256-bit big-endian unsigned integer) is encoded as a base-44 positional numeral using the defined alphabet. The alphabet excludes visually ambiguous characters (I, O, 0, 1, l) to ensure unambiguous human reading.

Symbol	Definition
`Σ₄₄`	44-character encoding alphabet — uppercase, digits, lowercase, excluding confusable chars
`BigInt(uScribeId)`	uScribeId interpreted as a 256-bit big-endian unsigned integer
`base=44`	Numeral base — each position represents one of 44 possible values

SPONGE-X586 ALTERNATIVE HASH (ALTERNATIVE EMBODIMENT)

```
State width: w = 576 bits
Rate: r = w − 2c (capacity c)
Output: 256 bits (32 bytes)
hashAlg field: 0x02
```

The SPONGE-X586 construction provides post-quantum collision resistance as an alternative to SHA-256. It uses a sponge construction with a 576-bit permutation width, absorbing input at rate r bits per permutation and squeezing 256 bits of output.

Symbol	Definition
`w`	Permutation width — 576 bits
`c`	Capacity — security parameter controlling collision resistance
`r`	Rate — bits absorbed per permutation: r = 576 − 2c
`0x02`	hashAlg field value identifying SPONGE-X586 in Capsule header

2.2 Sequential Ordinal Targeting (SOT)

SATOSHI LOCUS DEFINITION

```
Locus = (txid, vout, offset)
Lead Satoshi Locus = (txid, vout, 0)
Outpoint = (txid, vout)
```

A locus uniquely identifies a single satoshi within a specific UTXO. The txid is the Bitcoin transaction identifier, vout is the output index within that transaction, and offset is the zero-based satoshi index within the output's value range. The Lead Satoshi is always at offset 0.

Symbol	Definition
`txid`	32-byte Bitcoin transaction identifier — uniquely identifies the containing transaction
`vout`	Output index — integer identifying which output of the transaction contains the satoshi
`offset`	Satoshi position within the UTXO value range — 0 for Lead Satoshi
`Outpoint`	The (txid, vout) pair — standard Bitcoin UTXO reference

DETERMINISTIC LOCUS SELECTION FUNCTION

```
Sort candidates by:
(confirmations DESC, amount ASC,
txid lexicographic ASC, vout ASC)
Select: first candidate with locus ∉ locus_registry
Bind: locus_registry[locus] ← uScribeId
```

The selection function is deterministic: given the same UTXO set and registry state, all implementations select the same target locus. The availability check ensures no uScribeId is bound to a previously claimed locus within the application namespace.

Symbol	Definition
`locus_registry`	Persistent database mapping locus → uScribeId for all prior bindings
`confirmations`	Number of Bitcoin block confirmations — prefers older, more stable UTXOs
`amount`	UTXO value in satoshis — prefers smaller UTXOs to preserve high-value outputs
`locus ∉ locus_registry`	Availability constraint — locus must not already be bound

TAPROOT COMMITMENT (ALTERNATIVE EMBODIMENT)

```
t = H(internal_key ‖ uScribeId)
P_tweaked = P_internal + t·G
```

In the Taproot alternative embodiment, the uScribeId is committed in the Bitcoin transaction itself via a Taproot tweak. This provides a cryptographically verifiable on-chain link between the transaction and the uScribeId provable to any Bitcoin node.

Symbol	Definition
t	Taproot tweak — hash of the concatenation of the internal key and uScribeId
P_internal	Internal public key — base Taproot key before tweak
G	Secp256k1 generator point
P_tweaked	Tweaked public key — the observable on-chain public key including the commitment
‖	Concatenation operator

2.3 Cross-Chain Zero-Entropy Parity Engine (CZEPE)

LATTICE MAPPING Φ

```
Φ : D → P ∈ □ ⊂ ℝ^N
Basis matrix: B ∈ ℝ^{N×N}, |det(B)| = 1
H(P) = H(D) (information preservation)
```

The input data state D is mapped to a coordinate P in a high-dimensional lattice L via the bijective mapping Φ. The unit-determinant constraint on the basis matrix B ensures Φ is a volume-preserving bijection, guaranteeing that no information is lost in the mapping — the Shannon entropy of P equals that of D.

Symbol	Definition
D	Input data state — any digital artifact to be anchored and bridged
P	Lattice coordinate in $\mathbb{R}^N$ — geometric representation of D preserving all relational structure
□	High-dimensional lattice — □ ⊂ $\mathbb{R}^N$ with basis matrix B
N	Lattice dimensionality — N=1024 in preferred embodiment
B	Lattice basis matrix — |det(B)|=1 ensures bijection and entropy preservation
H(·)	Shannon entropy functional

LATTICE COLLAPSE FUNCTION Ψ

```
Ψ(P, K) → σ ∈ {0,1}^{256}
σ = 32-byte persistent seed
γ(|Ψ(P₁,K) − Ψ(P₂,K)| < ε) ≈ 0 (collision resistance)
```

The lattice collapse function Ψ deterministically maps the lattice coordinate P and system parameter set K to a 32-byte persistent seed σ. The function is implemented using SPONGE-X586 applied to the serialized lattice coordinate. It is collision-resistant: the probability of two distinct data states producing the same seed is negligible.

Symbol	Definition
σ	32-byte persistent seed — compact representation of the full data state D for cross-chain use
K	System parameter set — governs collapse function stability and security bounds
P	Input lattice coordinate from Φ(D)
Collision resistance	Prob[$\Psi(D_1,K)=\Psi(D_2,K)$ for $D_1 \neq D_2$] is negligible under security parameters

ZERO-ENTROPY PARITY THEOREM

```
γ(σ_α, K) ≡ γ(σ_β, K) ≡ D
ΔH = H(D_manifest) − H(D_original) = 0.0000
Proof: H(P) = H(D) [bijectivity of Φ]
H(γ(σ,K)) = H(D) [left-inverse of Ψ]
∴ H(D_manifest) = H(D_original) ∴ ΔH = 0 ✓
```

The Zero-Entropy Parity Theorem proves that the data state D can be reconstructed identically from either chain's inscription using the rehydration function Γ. Because Φ is bijective and Γ is the left-inverse of Ψ, no information is lost at any stage of the encode → inscribe → bridge → rehydrate pipeline.

▶ ZERO-ENTROPY PARITY THEOREM: For any data state D encoded and bridged through the CZEPE pipeline, ΔH = H(D_manifest) − H(D_original) = 0. The cross-chain bridge is lossless. □

Symbol	Definition
σ_α	Seed inscribed on Chain_α (XRP Ledger in preferred embodiment)
σ_β	Seed inscribed on Chain_β (Stellar Network in preferred embodiment)
γ(σ,K)	Rehydration function — reconstructs D from seed σ and parameter set K
ΔH	Entropy differential — proven to equal 0.0000 by the bijectivity chain above
D_manifest	Data state reconstructed from cross-chain inscription
D_original	Original input data state before encoding

2.4 Alternative Embodiment Formulas

DISTRIBUTED AFFINITY SCORE Ψ_NET (DISTRIBUTED EMBODIMENT)

$$\Psi_{net}(i,j) = \Psi(i,j) - \eta \cdot latency(n_i, n_j)$$

In the distributed embodiment, the affinity score is penalized by network latency between the requesting node n_i and the target agent node n_j. The penalty coefficient η controls the tradeoff between semantic alignment and network locality.

Symbol	Definition
Ψ(i,j)	Base affinity score from the SV Routing formula
η	Latency penalty coefficient — higher η favors network-local routing
latency(n_i,n_j)	Measured or estimated round-trip latency between nodes n_i and n_j

FEDERATED E-DIFFERENTIAL PRIVACY NOISE (FEDERATED EMBODIMENT)

$$\eta \sim \mathbb{N}(0, \sigma^2 \cdot I)$$

$$Mechanism: M(s_i) = s_i + \eta$$

In the federated embodiment, Gaussian noise η is injected into thread semantic vectors before cross-federation routing to provide ε-differential privacy. The noise scale σ² is calibrated to the required privacy budget ε.

Symbol	Definition
η	Gaussian noise vector drawn from N(0, σ²·I)
σ^2	Noise variance — calibrated to privacy budget ε per the Gaussian mechanism
I	Identity matrix — noise is isotropic across all d embedding dimensions
ε	Privacy budget parameter — smaller ε = stronger privacy, larger noise
M(s_i)	Privacy-protected semantic vector — noisy version of s_i for cross-federation use

2.5 End-to-End Pipeline Identity

FULL PIPELINE COMPOSITION

```
Step 1 — Encode: Capsule(D) → uScribeId = SHA-256(capsuleBytes)
Step 2 — Anchor: SOT(uScribeId) → (txid, vout, 0) on Bitcoin
Step 3 — Bridge: Φ(D) → P → Ψ(P,K) → σ
Inscribe(σ) on Chain_α and Chain_β
Step 4 — Settle: LSL_event → pacs.008 XML → banking rail

Invariant: γ(σ_α, K) ≡ γ(σ_β, K) ≡ D and ΔH = 0
```

The complete Lone Star Ledger pipeline is a composition of four deterministic, entropy-preserving transformations. The end-to-end invariant guarantees that the original data state D is recoverable from any chain inscription with zero information loss, while the Bitcoin satoshi locus and pacs.008 settlement record provide independent, auditable provenance at every layer.